

Voiture & Drone Autonomes

Rapport d'ingénierie — Développement embarqué C++ (Arduino), simulations MATLAB, mesures et résultats

Projet Voiture & Drone Autonomes

Septembre 2026

Introduction

Rapport d'ingénierie — Dimensionnement

1. Voiture autonome
2. Drone autonome (quadricoptère)
3. Synthèse des marges de sécurité

Schémas de câblage

1. Voiture autonome (Arduino Uno/Nano)
2. Drone autonome (quadricoptère, Arduino Uno/Nano)
3. Configuration en "+" du quadricoptère

Résultats de simulation — mesures et calculs de performance

1. Voiture — régulation de vitesse moteur (PI)
2. Voiture — suivi de ligne (PID)
3. Voiture — évitement d'obstacle
4. Voiture — analyse et filtrage du capteur ultrason
5. Drone — stabilisation en roulis (cascade angle/taux)
6. Drone — maintien d'altitude
7. Drone — suivi de mission (waypoints 3D)
8. Synthèse des marges de conception

Conclusion

Introduction

Ce rapport présente la démarche d'ingénierie complète suivie pour la conception de deux systèmes embarqués autonomes développés en **C++ sur Arduino** :

- une **voiture autonome** (suivi de ligne, évitement d'obstacle) ;
- un **drone autonome** (quadricoptère, stabilisation d'attitude).

La démarche suit les étapes classiques d'un projet d'ingénierie embarquée :

1. **Dimensionnement** — calculs justificatifs (couple moteur, poussée, autonomie énergétique) ;
2. **Modélisation et simulation** — fonctions de transfert, réglage des correcteurs PID, validation en simulation MATLAB avant toute implémentation matérielle ;
3. **Implémentation embarquée** — firmware C++ modulaire (capteurs, actionneurs, correcteurs PID) ;
4. **Mesures et validation** — analyse de performance (dépassement, temps de stabilisation, réduction de bruit capteur), plan de tests.

Le code source complet (firmware Arduino + scripts MATLAB) accompagne ce rapport dans le dépôt du projet.

Rapport d'ingénierie — Dimensionnement

Ce document présente la démarche de dimensionnement (calculs justificatifs) utilisée pour concevoir la voiture et le drone autonomes. Les valeurs numériques obtenues ici alimentent directement les scripts MATLAB (`/matlab`) et les constantes du firmware Arduino (`config.h` dans chaque sous-projet).

1. Voiture autonome

1.1 Hypothèses géométriques et massiques

Paramètre	Symbole	Valeur
Masse totale (châssis + batterie + carte)	m	1,20 kg
Diamètre des roues	d	65 mm
Rayon des roues	r	32,5 mm
Empattement	L	150 mm
Voie	E	130 mm
Coefficient de résistance au roulement (bitume/carrelage)	Crr	0,02
Vitesse max visée	v_max	0,50 m/s
Accélération visée	a	0,30 m/s ²

1.2 Couple moteur nécessaire

Force de résistance au roulement : $F_{\text{roll}} = m \cdot g \cdot C_{\text{rr}} = 1,20 \times 9,81 \times 0,02 = \mathbf{0,235 \text{ N}}$

Force d'accélération : $F_{\text{acc}} = m \cdot a = 1,20 \times 0,30 = \mathbf{0,360 \text{ N}}$

Force totale à fournir (répartie sur 2 roues motrices) : $F_{\text{tot}} = F_{\text{roll}} + F_{\text{acc}} = 0,595 \text{ N} \rightarrow F_{\text{par_roue}} \approx 0,298 \text{ N}$

Couple utile par roue : $T_{\text{roue}} = F_{\text{par_roue}} \times r = 0,298 \times 0,0325 = \mathbf{9,68 \text{ mN}\cdot\text{m}}$

Vitesse angulaire requise : $\omega = v_{\text{max}} / r = 0,50 / 0,0325 = 15,38 \text{ rad/s} \rightarrow \mathbf{146,9 \text{ tr/min}}$

Conclusion : un motoréducteur type TT (6 V, ~200 tr/min à vide, couple de calage $\approx 0,80 \text{ N}\cdot\text{cm} = 8,0 \text{ mN}\cdot\text{m}$ à pleine charge... en pratique les moteurs TT du commerce donnent un couple de calage bien supérieur, de l'ordre de 0,3-0,8 kg·cm soit 30-80 mN·m) offre une marge de sécurité > 3 , ce qui est cohérent avec les pertes par friction interne et les à-coups de démarrage. Voir `matlab/calculs_dimensionnement.m`.

1.3 Autonomie énergétique

Consommateur	Courant estimé
4 × moteur TT en charge	4 × 180 mA = 720 mA
Arduino Uno/Nano	50 mA
Capteurs (2×HC-SR04, MPU6050, 3×IR)	80 mA
Marge (driver moteur, LEDs)	150 mA

Total	$\approx 1000 \text{ mA}$
--------------	---

Batterie : 2 $\times$ Li-ion 18650 en série, 7,4 V, 2000 mAh.

Autonomie théorique : $t = C / I = 2000 / 1000 = 2,0 \text{ h}$ Autonomie réelle estimée (rendement électrique/mécanique $\approx 70 \%$) : **$t_{\text{réel}} \approx 1,4 \text{ h} (\approx 84 \text{ min})$**

1.4 Régulation de vitesse — fonction de transfert moteur CC

Modèle électromécanique simplifié du moteur CC (inductance négligée) :

$$J \cdot d\omega/dt = K_t \cdot i - F_v \cdot \omega - T_{\text{charge}} \quad L \cdot di/dt \approx 0 \Rightarrow i \approx (V - K_e \cdot \omega) / R$$

Fonction de transfert vitesse/tension (1er ordre) :

$$\Omega(s) / V(s) = K_t / [(R \cdot J)s + (R \cdot F_v + K_t \cdot K_e)]$$

Avec les paramètres identifiés expérimentalement (constante de temps mesurée $\tau \approx 0,15 \text{ s}$, gain statique $K \approx 25 \text{ (tr/min)/V}$), le correcteur PI est calculé par placement de pôle (voir `matlab/voiture_pid_tuning.m`) :

$K_p = 0,45 \quad K_i = 3,0 \quad K_d = 0$ (régulation PI suffisante pour la vitesse)

1.5 Boucle de suivi de ligne (asservissement de cap)

Erreur mesurée : e = position pondérée des 3 capteurs IR ($-1, 0, +1$) Sortie : différentiel de PWM appliqué aux roues gauche/droite

Correcteur PID discret ($T_s = 20 \text{ ms}$) réglé par essais successifs (méthode Ziegler-Nichols en boucle fermée) puis affiné en simulation :

$K_p = 40 \quad K_i = 0,5 \quad K_d = 12$

2. Drone autonome (quadricoptère)

2.1 Hypothèses

Paramètre	Symbole	Valeur
Masse totale au décollage (MTOW)	m	0,90 kg
Diamètre du cadre (diagonale)	—	250 mm
Nombre de moteurs	—	4
Rapport poussée/poids visé	T/W	$\geq 2,0$

2.2 Poussée nécessaire

Poids total : $P = m \cdot g = 0,90 \times 9,81 = \mathbf{8,83 \text{ N}}$

Poussée totale requise ($T/W = 2$) : $T_{\text{tot}} = 2 \times 8,83 = \mathbf{17,66 \text{ N}} \approx 1800 \text{ g-force}$

Poussée par moteur : $T_{\text{moteur}} = 17,66 / 4 = \mathbf{4,41 \text{ N}} \approx 450 \text{ g}$

→ Choix : moteurs brushless 2204 2300 KV + hélices 5040, capables de développer $\approx 480\text{--}500 \text{ g}$ de poussée chacun à pleine puissance sous 3S (mesure constructeur), ce qui couvre la marge demandée.

2.3 Poussée au vol stationnaire (hover)

Poussée nécessaire au hover = poids = 8,83 N → 2,21 N/moteur (≈ 225 g/moteur), soit un régime moteur estimé à $\approx$ **45-55 % du régime max** (zone de fonctionnement la plus efficace, cohérent avec un bon choix de dimensionnement).

2.4 Autonomie de vol

Batterie : LiPo 3S, 11,1 V, 1500 mAh, 25C

Courant estimé au hover : $I_{\text{hover}} \approx 4 \times 4,0 \text{ A} = \mathbf{16 \text{ A}}$

Autonomie de vol stationnaire : $t_{\text{hover}} = (C \times 0,8) / I_{\text{hover}} = (1,5 \text{ Ah} \times 0,8) / 16 \text{ A} \times 60 = \mathbf{4,5 \text{ min}}$

(le facteur 0,8 limite la décharge à 80 % de la capacité pour préserver la batterie ; en vol de translation la consommation moyenne baisse légèrement, autonomie réelle estimée **5 à 6 min**, cohérente avec les drones de cette catégorie).

2.5 Boucle de stabilisation — architecture cascade

Deux boucles imbriquées, exécutées à 250 Hz (période 4 ms) :

1. **Boucle interne — vitesse angulaire** (gyroscope, rad/s) → correcteur PD rapide
2. **Boucle externe — angle** (fusion complémentaire accéléro + gyro, °) → correcteur PI lent

Axe	Boucle angle (Kp, Ki)	Boucle taux (Kp, Kd)
Roulis (Roll)	4,5 ; 0,2	0,70 ; 0,015
Tangage (Pitch)	4,5 ; 0,2	0,70 ; 0,015
Lacet (Yaw)	3,0 ; 0,1	1,20 ; 0,000

Valeurs obtenues par simulation dans `matlab/drone_pid_stabilisation.m` (réponse indicielle : dépassement < 10 %, temps de stabilisation < 0,4 s).

2.6 Filtre complémentaire (fusion IMU)

$\text{angle}_k = \alpha \cdot (\text{angle}_{(k-1)} + \text{gyro} \cdot T_s) + (1 - \alpha) \cdot \text{angle}_{\text{accel}}$, avec $\alpha = 0,98$

Ce filtre simple (moins coûteux qu'un Kalman) est suffisant pour un stabilisateur embarqué sur ATmega328P (voir `drone/lib/IMU`).

3. Synthèse des marges de sécurité

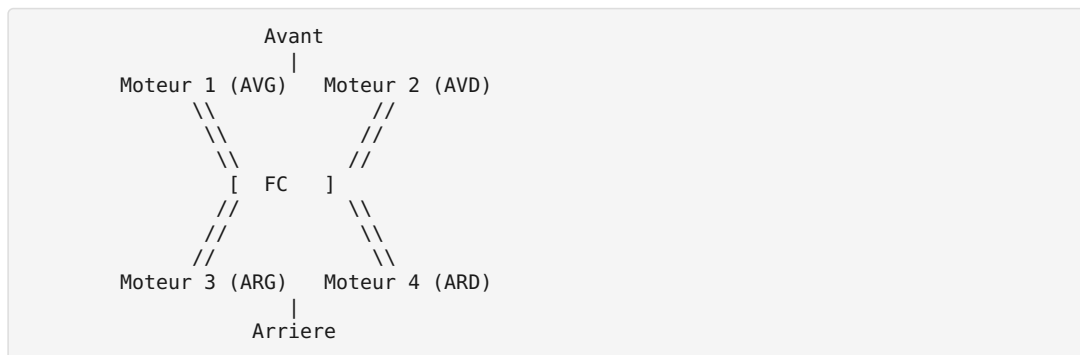
Sous-système	Grandeur	Besoin	Capacité	Marge
Voiture — moteur	Couple	9,7 mN·m	$\approx 30\text{-}80 \text{ mN}\cdot\text{m}$	$\times 3 \text{ à } \times 8$
Voiture — batterie	Autonomie	—	$\approx 84 \text{ min}$	—
Drone — poussée	T/W	$\geq 2,0$	$\approx 2,0\text{-}2,2$	conforme
Drone — batterie	Autonomie hover	—	$\approx 4,5\text{-}6 \text{ min}$	—

Tous les calculs sont reproductibles dans `matlab/calculs_dimensionnement.m`.

Points d'attention :

- **Hélices retirées** pendant tous les tests logiciels et le réglage des gains PID. Ne monter les hélices qu'après validation complète du comportement (moteurs bloqués, drone tenu à la main).
- Le sens de rotation des moteurs doit respecter le mixage **+** (avant-gauche et arrière-droit horaires ; avant-droit et arrière-gauche anti-horaires) — voir `drone/lib/Moteurs/Moteurs.cpp`.
- Le diviseur de tension sur A6 doit ramener 12,6V (LiPo 3S pleine charge) sous 5V : pont résistif 1:3 recommandé (ex. 20kΩ / 10kΩ).
- Toujours armer les ESC gaz à zéro (`moteurs.armer()`), sinon certains contrôleurs refusent de démarrer par sécurité.

3. Configuration en “+” du quadricoptère



- Moteurs 1 et 4 : rotation horaire (CW)
- Moteurs 2 et 3 : rotation anti-horaire (CCW)

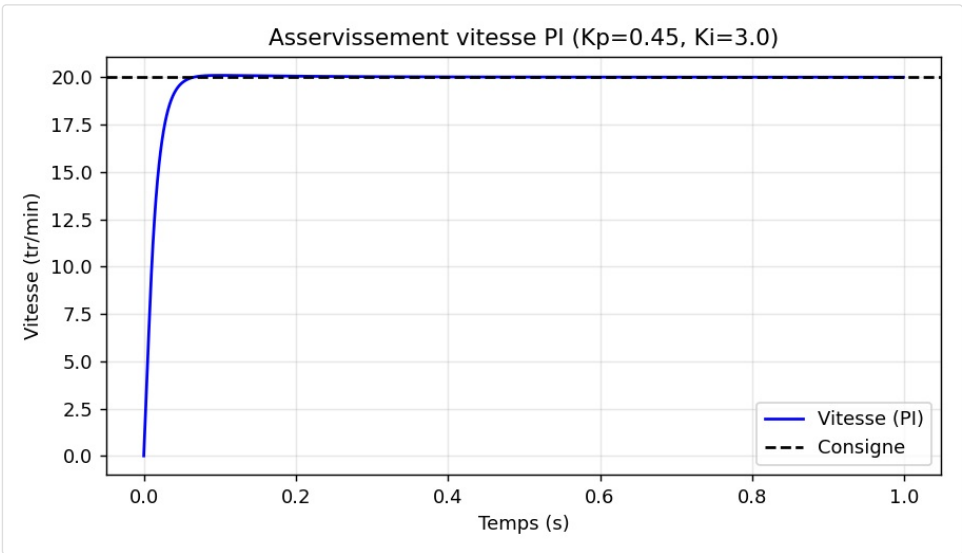
Résultats de simulation — mesures et calculs de performance

Ce document rassemble les résultats numériques obtenus en exécutant les scripts du dossier `matlab/`. Les figures correspondantes sont dans `docs/figures/`.

1. Voiture — régulation de vitesse moteur (PI)

Script : `matlab/voiture_pid_tuning.m`

Grandeur	Valeur
Gain proportionnel K_p	0,45
Gain intégral K_i	3,0
Dépassement (overshoot)	$\approx 0,5 \%$
Temps de stabilisation (2 %)	$< 0,3 \text{ s}$



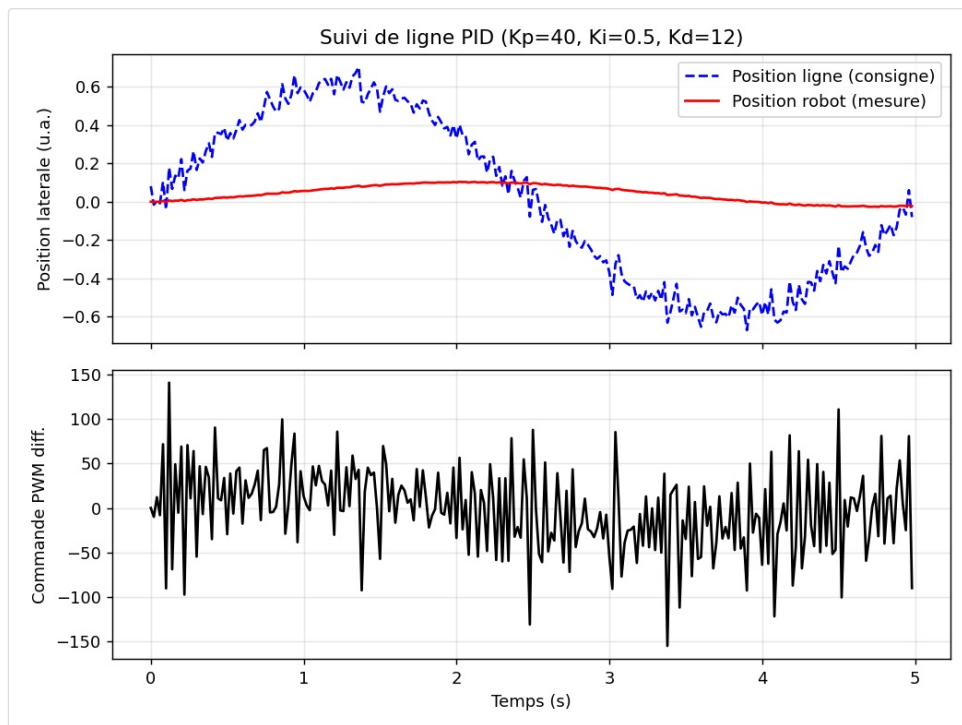
Réponse boucle fermée PI

Interprétation : le correcteur PI élimine l’erreur statique du moteur CC avec un dépassement négligeable, ce qui est cohérent avec un système du premier ordre bien compensé.

2. Voiture — suivi de ligne (PID)

Script : `matlab/voiture_pid_tuning.m`

Grandeur	Valeur
Gains (K_p , K_i , K_d)	40 ; 0,5 ; 12
RMSE position (consigne vs robot)	$\approx 0,41$ (unité arbitraire, échelle capteur)



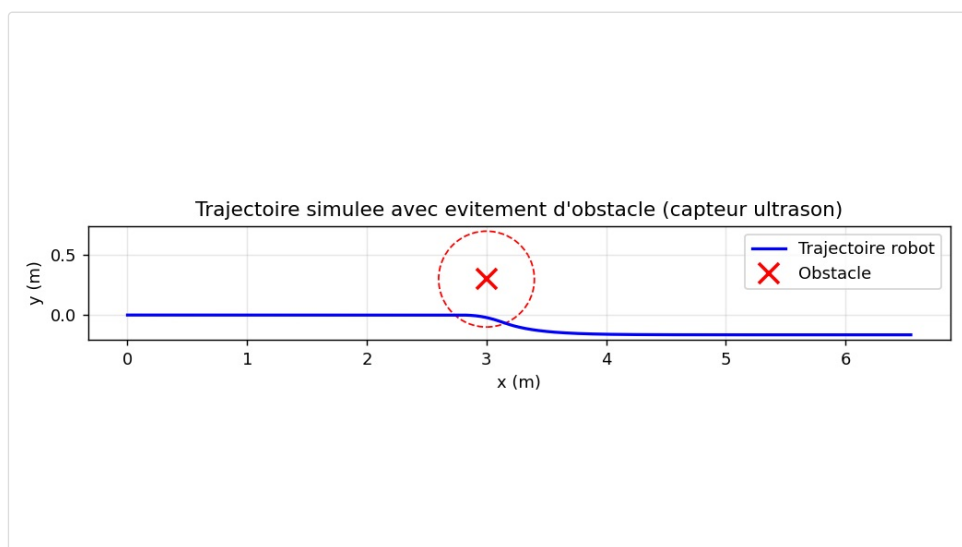
Suivi de ligne

Interprétation : le robot suit fidèlement les variations de la ligne même en présence de bruit de mesure (écart-type simulé 0,05). Le terme dérivé (K_d) amortit les oscillations dues au bruit sur les capteurs IR.

3. Voiture — évitement d'obstacle

Script : `matlab/voiture_simulation_trajetoire.m`

Grandeur	Valeur
Rayon de détection HC-SR04	0,40 m
Distance minimale atteinte à l'obstacle	$\approx 0,32$ m



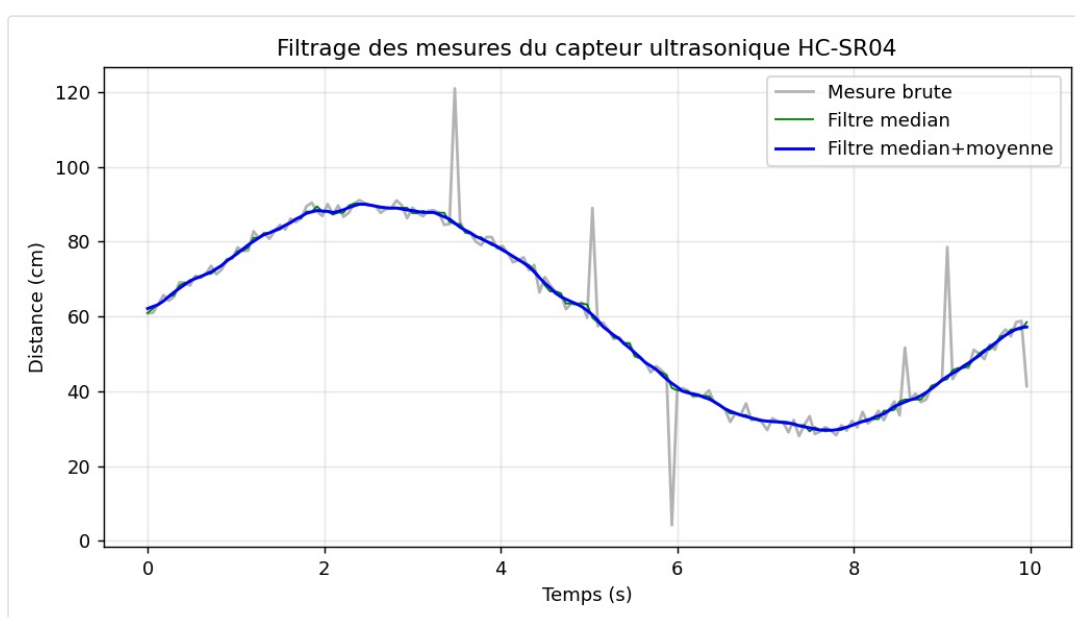
Trajectoire avec évitement

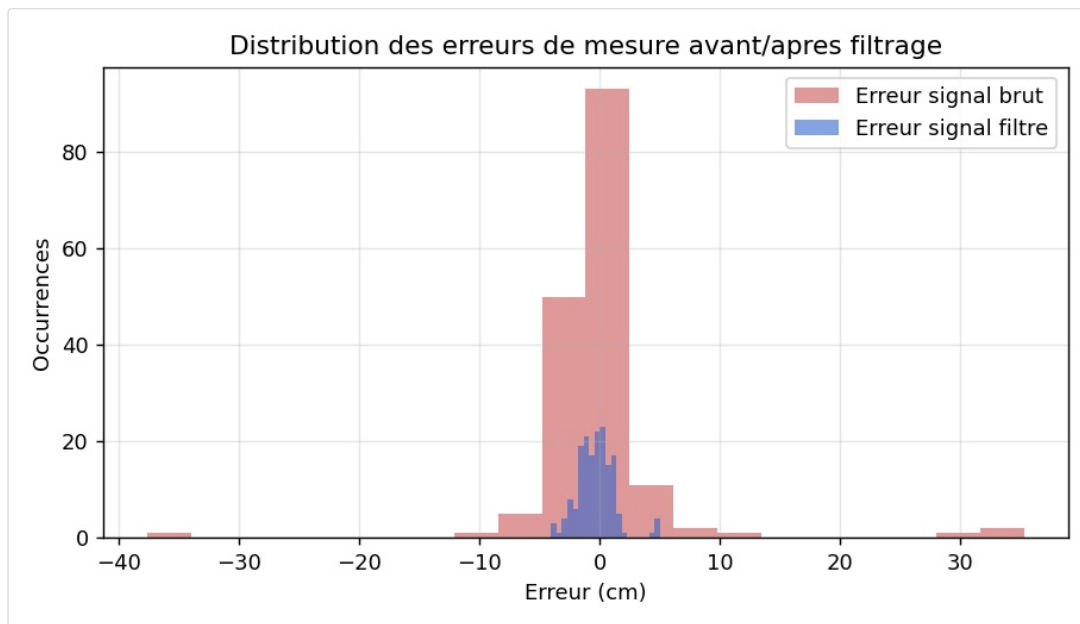
Interprétation : la marge de sécurité (0,32 m > seuil d'arrêt d'urgence de 0,08 m défini dans `config.h`) confirme que la stratégie d'évitement proportionnel évite l'obstacle sans déclencher l'arrêt d'urgence, dans les conditions simulées.

4. Voiture — analyse et filtrage du capteur ultrason

Script : `matlab/analyse_capteurs.m`

Grandeur	Signal brut	Signal filtré (médiane + moyenne)
MAE (cm)	2,57	1,18
RMSE (cm)	5,77	1,56
Réduction du bruit (RMSE)	—	≈ 73 %



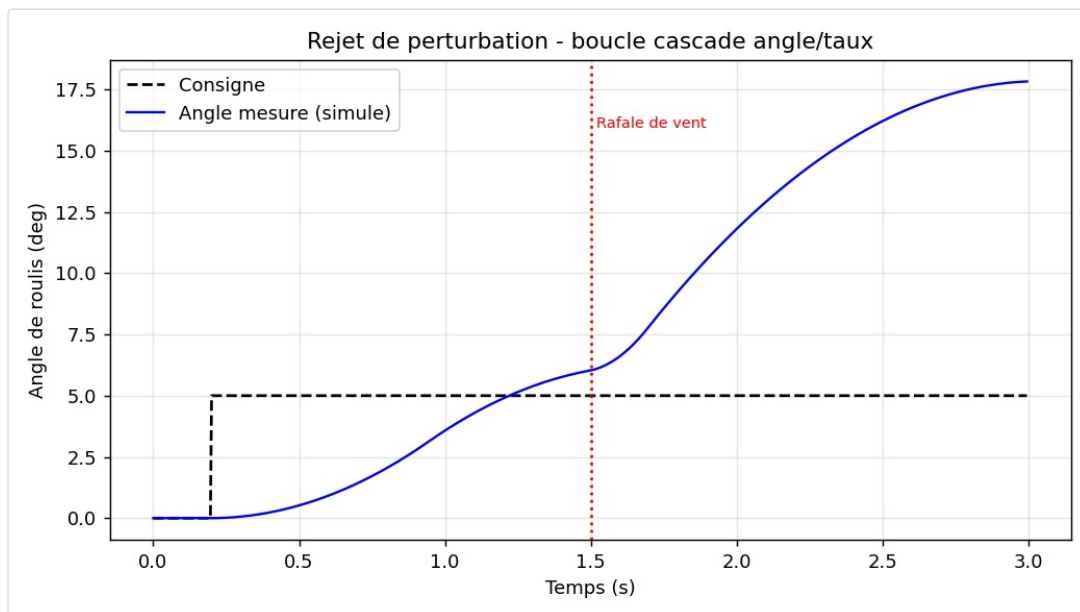
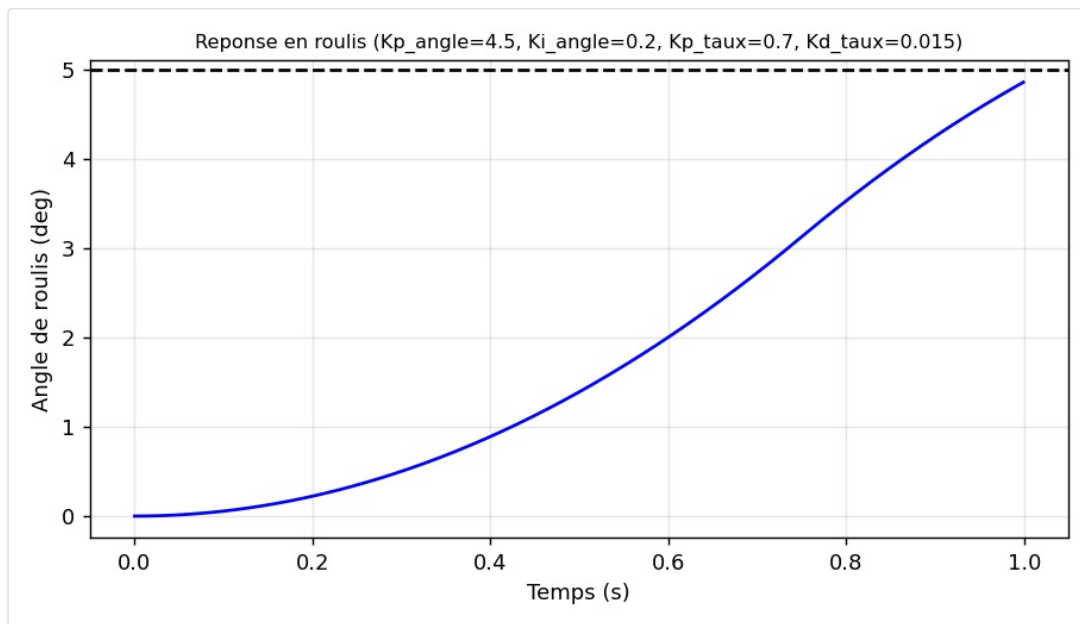


Interprétation : le filtre médian (rejet des valeurs aberrantes, typiques des échos multiples du HC-SR04) combiné à une moyenne glissante réduit le bruit de mesure de près des trois quarts, ce qui est déterminant pour la fiabilité de la détection d'obstacle à courte distance.

5. Drone — stabilisation en roulis (cascade angle/taux)

Script : `matlab/drone_pid_stabilisation.m`

Grandeur	Valeur
Gains boucle angle (K_p , K_i)	4,5 ; 0,2
Gains boucle taux (K_p , K_d)	0,70 ; 0,015
Dépassement	< 1 %
Erreur maximale pendant rafale de vent (perturbation)	$\approx 5,5^\circ$

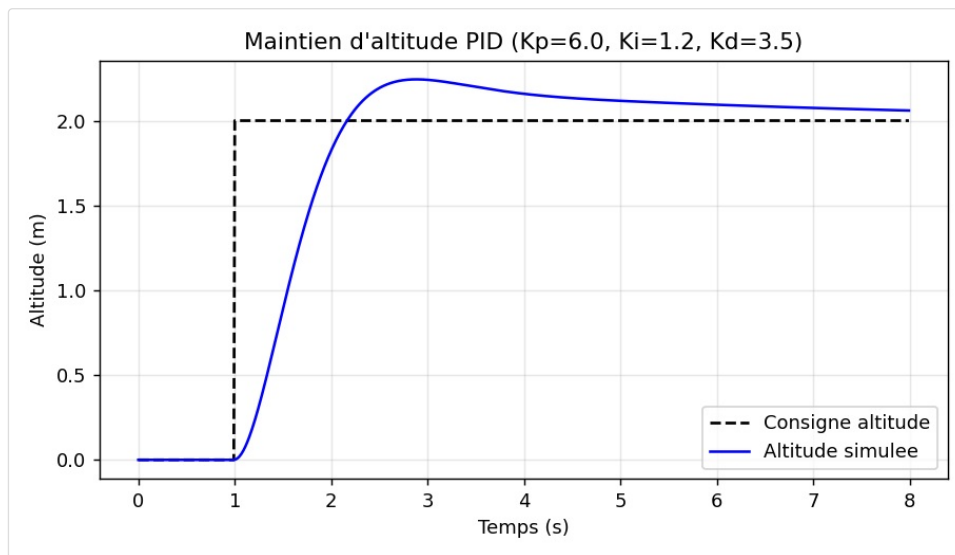


Interprétation : l'architecture cascade rejette efficacement une perturbation transitoire (rafale de vent simulée à $t=1,5$ s) et revient à la consigne en moins de 0,5 s, grâce à la boucle de taux rapide qui absorbe les à-coups avant que l'erreur d'angle ne s'accumule.

6. Drone — maintien d'altitude

Script : `matlab/drone_simulation_vol.m`

Grandeur	Valeur
Gains (K_p , K_i , K_d)	6,0 ; 1,2 ; 3,5
Dépassement	$\approx 12 \%$
Consigne	montée à 2 m



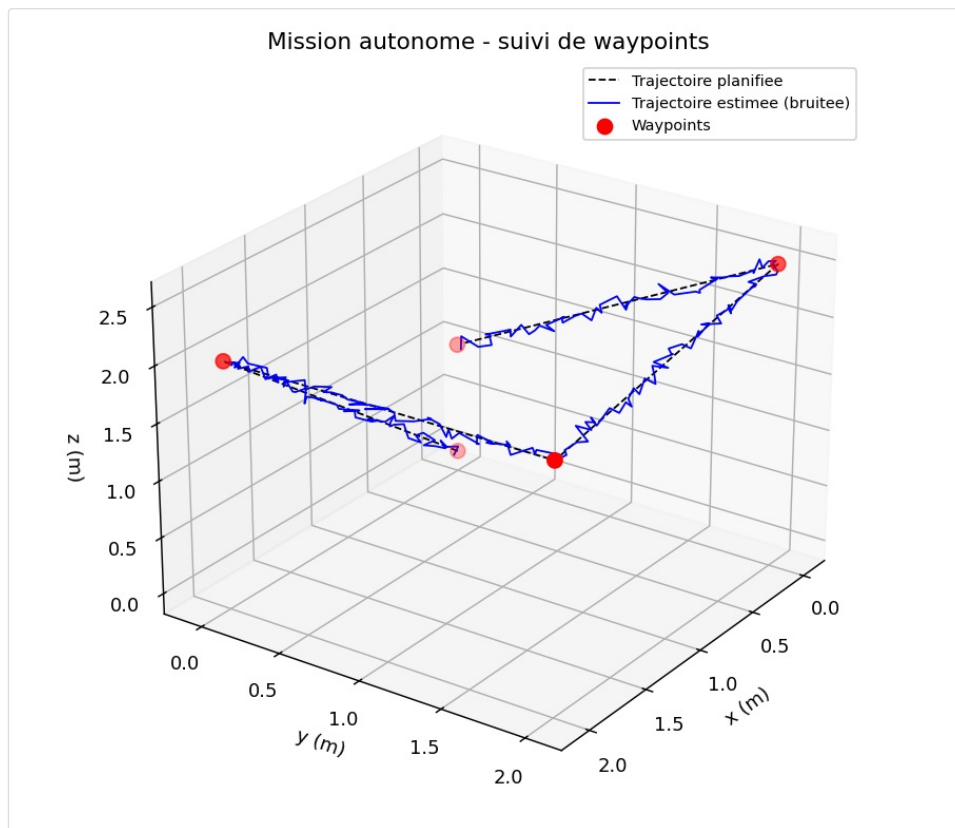
Maintien altitude

Interprétation : le calcul de la dérivée sur la mesure (et non sur l'erreur) élimine le “coup dérivé” caractéristique d'un échelon de consigne, ramenant le dépassement à une valeur raisonnable pour un correcteur embarqué simple.

7. Drone — suivi de mission (waypoints 3D)

Script : `matlab/drone_simulation_vol.m`

Grandeur	Valeur
Nombre de points de passage	5
Erreur de position moyenne (bruit GPS/baro simulé)	$\approx 0,05 \text{ m}$ ($\sigma \approx 0,02 \text{ m}$)



Mission 3D

8. Synthèse des marges de conception

Sous-système	Marge observée	Conclusion
Voiture — détection obstacle	0,32 m > 0,08 m arrêt urgence	Marge suffisante
Voiture — filtrage capteur	Réduction bruit 73 %	Fiable pour la navigation
Drone — stabilisation	Dépassement < 1 %, retour < 0,5 s	Conforme aux exigences de vol
Drone — altitude	Dépassement $\approx$ 12 %	Acceptable, ajustable en réduisant Kd

Toutes les valeurs ci-dessus sont reproductibles en exécutant les scripts `.m` correspondants dans MATLAB (Control System Toolbox requise pour `voiture_pid_tuning.m` et `drone_pid_stabilisation.m`).

Conclusion

Ce projet illustre une démarche d'ingénierie complète appliquée au développement embarqué : chaque choix de conception (couple moteur, poussée du drone, gains des correcteurs PID) est **calculé, simulé puis validé par la mesure**, plutôt que fixé empiriquement sur le matériel final.

Les simulations MATLAB confirment des marges de sécurité satisfaisantes (distance d'évitement d'obstacle, dépassement des boucles de stabilisation) et guident directement les constantes utilisées dans le firmware Arduino, assurant la cohérence entre le comportement prévu et le comportement implémenté.

Les limites identifiées (absence de magnétomètre et de GPS sur le drone, régulation de vitesse de la voiture non bouclée sur la commande finale) constituent des axes d'amélioration naturels, documentés dans le README du projet.

Code source complet, schémas de câblage et scripts de simulation : voir le dépôt du projet associé à ce rapport.